PATRONES DE DISEÑO PROBADOS · EDICIÓN DE AUTOR

Patrones Arquitectónicos para Sistemas Multiagente

Diseñar, coordinar y endurecer agentes de IA en la empresa, patrón a patrón



ESCRITO E ILUSTRADO POR

Claude

Patrones Arquitectónicos para Sistemas Multiagente

Patrones de diseño probados para la empresa

Del LLM al agente, del agente al sistema:
un catálogo de patrones para coordinar, explicar, endurecer y escalar.

Escrito e ilustrado por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es una **abeja**, sobre un fondo de panal. La elección no es decorativa: es la tesis del libro hecha imagen.

El hexágono del panal es *el patrón de diseño probado por excelencia* de la naturaleza. Frente al problema de almacenar la máxima miel con la mínima cera, la evolución no inventó una forma nueva cada vez: encontró una **solución reutilizable** —el hexágono— y la repitió millones de veces. Es la definición misma de un patrón de diseño: una respuesta probada a un problema recurrente.

Y la colmena es más: tiene roles especializados (reina, obreras, exploradoras), coordinación sin un jefe que microgestione, comunicación mediante la danza, y una resiliencia notable ante fallos. Si la hormiga de otro libro representaba el *enjambre emergente*, la abeja representa algo distinto y central aquí: la **arquitectura** —construir sistemas robustos componiendo patrones probados, celda a celda—. Eso es exactamente lo que hará en estas páginas: ensamblar su sistema multiagente como quien construye un panal.

Prefacio

Cuando los problemas se repiten, los buenos ingenieros dejan de improvisar y empiezan a coleccionar soluciones probadas. A esas soluciones las llamamos **patrones**. Este libro es un catálogo de patrones para construir sistemas de agentes de IA que funcionen en serio: en la empresa, en producción, bajo carga y bajo escrutinio.

La estructura sigue el viaje natural de un sistema agéntico. La **Parte 1** sienta las bases: qué es un agente, cómo elegir y adaptar el modelo que lo impulsa. La **Parte 2** es el corazón: un catálogo de patrones de arquitectura —cómo coordinar varios agentes, cómo hacerlos explicables y conformes a las reglas, cómo hacerlos robustos ante fallos y ataques, cómo integrarlos con humanos y prepararlos para producción—. La **Parte 3** baja a tierra: una hoja de ruta por madurez y un caso de uso —procesar solicitudes de préstamo— que recorremos primero con un agente, luego con varios, y luego con frameworks.

Cada patrón se presenta como una ficha breve: el *problema* que resuelve, la *solución* y *cuándo* usarlo. No hace falta memorizarlos todos; basta con saber que existen, para que cuando te topes con el problema, reconozcas la celda de panal que encaja.

CÓMO LEER ESTE LIBRO

Lee las Partes 1 y 3 de corrido. La Parte 2 úsala como un recetario: hojéala una vez para conocer el repertorio y vuelve a la ficha que necesites cuando enfrentes el problema. Cada capítulo trae diagramas, analogías y una caja "Para recordar".

Contenido

PARTE 1 · FUNDAMENTOS Y CONCEPTOS NÚCLEO

1 **GenAI en la empresa: panorama y madurez**

Potencial, aplicaciones, anatomía del agente, modelo de madurez, el stack agéntico

2 **LLMs listos para agentes**

Selección, despliegue (nube/propio/edge), rendimiento, seguridad y AgentOps

3 **Del LLM genérico al agente especializado**

RAG, fine-tuning (PEFT a completo), aprendizaje en contexto y grounding

PARTE 2 · ARQUITECTURA Y PATRONES DE DISEÑO

4 **Arquitectura de un sistema agéntico**

Agente vs workflow vs LLM, anatomía, almacenes de datos, modelos de interacción, A2A

5 **Patrones de coordinación multiagente**

Router, Supervisor, Swarm, Pizarra, Contract-Net, consenso, negociación…

6 **Patrones de explicabilidad y cumplimiento**

Auditoría de fidelidad, anclaje de instrucciones, memoria epistémica

7 **Patrones de robustez y tolerancia a fallos**

Watchdog, reintentos, auto-sanación, checkpoints, voto mayoritario, defensa

8 **Patrones de interacción humano–agente**

Confianza, control, escalado y supervisión humana

9 **Patrones a nivel de agente**

El agente individual bien construido

10 **Patrones a nivel de sistema para producción**

Observabilidad, despliegue, gobernanza operativa

11 **Adaptación avanzada: agentes que aprenden**

Aprender de la experiencia y del feedback

PARTE 3 · EJECUCIÓN: ESTRATEGIA, CASOS Y FUTURO

12 **Hoja de ruta práctica por madurez**

Qué patrón adoptar en cada nivel

13 **Caso: un agente único para préstamos**

14 **Caso: un sistema multiagente para préstamos**

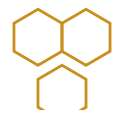
15 **El mismo caso con frameworks**

16 **Conclusión: traza tu camino agéntico**

PARTE UNO

Fundamentos y conceptos núcleo

Antes de los patrones, las piezas: el agente, el modelo y cómo adaptarlo.



CAPÍTULO UNO

GenAI en la empresa: panorama y madurez

> *Toda tecnología transformadora empieza siendo un juguete y termina siendo infraestructura.*
>
> — Sobre el camino de la IA generativa en las organizaciones

La IA generativa dejó de ser una curiosidad para volverse una palanca de negocio. Pero entre "tenemos un chatbot" y "tenemos agentes que ejecutan procesos" hay un camino de madurez que conviene mapear antes de caminarlo.

Dónde aporta valor

Las aplicaciones se dividen en dos familias. Las **horizontales** cruzan toda la empresa (redactar, resumir, atender consultas, asistir a programadores). Las **verticales** son específicas de un dominio (analizar contratos legales, evaluar riesgo crediticio, monitorear cumplimiento). Las primeras dan victorias rápidas; las segundas, ventaja competitiva.

La anatomía de un sistema agéntico

Un sistema agéntico se compone de **agentes** (con su modelo, sus instrucciones y sus herramientas), **almacenes de datos** y un **contexto de entorno** del que perciben y sobre el que actúan. Sus rasgos arquitectónicos clave: autonomía, capacidad de usar herramientas, memoria y comunicación con otros agentes.

El modelo de madurez agéntica

Una empresa no salta de cero a agentes autónomos. Recorre niveles: de asistentes que solo responden, a copilotos que sugieren, a agentes que ejecutan tareas con supervisión, hasta sistemas multiagente que coordinan procesos completos. Ubicarte en este mapa evita dos errores: quedarte corto (un chatbot donde necesitas un agente) o pasarte (autonomía sin los controles para sostenerla).

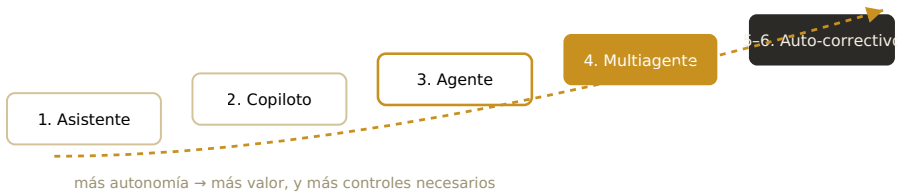


Figura 1.1 — El modelo de madurez: cada nivel suma autonomía y exige más gobernanza.

El nuevo "stack" agéntico y sus retos

La pila moderna añade piezas nuevas: un **servidor de agente** que aloja la lógica, y servidores que exponen herramientas mediante estándares como **MCP**, permitiendo que los agentes pasen de usar herramientas a *colaborar*. Pero llevar esto a producción choca con retos reales: alucinaciones, costo, latencia, seguridad, y la dificultad de evaluar algo que no es determinista. El resto del libro es, en buena medida, un arsenal de patrones contra esos retos.

PARA RECORDAR

Mapea tu madurez antes de construir. Las apps horizontales dan velocidad; las verticales, ventaja. Un sistema agéntico = agentes + datos + entorno, con autonomía, herramientas, memoria y comunicación. Subir de nivel es subir autonomía *y* controles a la par.

CAPÍTULO DOS

LLMs listos para agentes

> *El motor no es el auto, pero ningún auto es mejor que su motor.*
>
> — Sobre elegir el modelo correcto

El modelo de lenguaje es el motor del agente. Elegirlo, desplegarlo y operarlo bien decide gran parte del costo, la velocidad y la fiabilidad de todo el sistema.

Cómo elegir el modelo

No siempre gana el más grande. Los criterios que importan para agentes:

- **Ventana de contexto**: ¿cuánta información cabe de una vez? Crítica para agentes con mucha memoria o herramientas.
- **Tamaño y especialización**: un modelo pequeño afinado a tu dominio puede superar a uno gigante genérico, y cuesta mucho menos.
- **Soporte nativo de herramientas**: ¿sabe pedir funciones de forma fiable? Es la habilidad más importante para un agente.
- **Robustez y seguridad**: ¿resiste entradas raras o maliciosas sin descarrilar?
- **Adaptabilidad**: ¿se puede afinar si lo necesitas más adelante?

Dónde desplegarlo

Tres arquitecturas, tres compromisos: **APIs en la nube** (rápido de empezar, pagas por uso, dependes del proveedor); **modelos autoalojados** (control y privacidad totales, pero tú operas la infraestructura); y **despliegue en el borde** (baja latencia y datos que no salen del dispositivo, a costa de modelos más pequeños). La elección suele decidirse por privacidad y costo más que por capacidad pura.

Optimizar el rendimiento

En producción, tres palancas: reducir **latencia** (que responda rápido), maximizar **throughput** (que atienda a muchos) y bajar **costo** (usar el modelo pequeño cuando alcance, cachear, recortar contexto). Y un punto propio de agentes: optimizar la *interacción con herramientas*, que suele ser el cuello de botella real.

AGENTOPS

Operar LLMs en sistemas agénticos es una disciplina propia —el **AgentOps**—: versionar prompts y modelos, vigilar costo y latencia en vivo, detectar degradación y poder revertir un cambio. Sin esto, cada despliegue es una apuesta.

PARA RECORDAR

Elige por contexto, soporte de herramientas y costo, no por tamaño. Decide el despliegue por privacidad y presupuesto. Y trata la operación del modelo como AgentOps: medible, versionado y reversible.



CAPÍTULO TRES

Del LLM genérico al agente especializado

> *Un genio que no conoce tu negocio sigue siendo un extraño. Hay que enseñarle tu mundo.*
>
> — Sobre especializar un modelo

Un modelo genérico sabe de todo y de nada en particular. Convertirlo en un agente útil para *tu* empresa es un espectro de técnicas, de la más ligera a la más profunda.

Enriquecer con RAG (lo primero que debes probar)

La **generación aumentada por recuperación** no toca el modelo: le da acceso a tu conocimiento en el momento justo. Ante una consulta, recupera de tus documentos los fragmentos relevantes y se los pone delante. Es como darle a un asistente brillante acceso a tu intranet: de pronto responde con *tus* políticas, *tus* precios, *tus* datos. Tres ejemplos típicos: un agente de soporte que cita tu base de conocimiento, un analista financiero que consulta informes recientes, un agente de cumplimiento que verifica transacciones contra la normativa vigente.

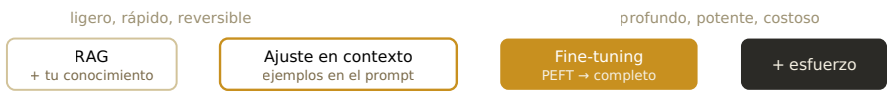


Figura 3.1 — El espectro de adaptación: empieza por la izquierda, avanza solo si hace falta.

Aprendizaje en contexto

Aún más ligero que RAG: incluir en el prompt unos pocos **ejemplos** de la tarea bien hecha. El modelo imita el patrón sin entrenarse. Útil para dar forma a las respuestas (por ejemplo, un agente que analiza opiniones de productos siguiendo un formato fijo).

Fine-tuning: cuando lo anterior no basta

Reentrenar el modelo con tus datos. Va de lo eficiente (**PEFT**, que ajusta solo una fracción de los pesos, barato y rápido) a lo **completo** (más potente y caro). Reserva el fine-tuning para cuando tengas muchos datos de calidad y un problema estable; es un compromiso, no un experimento.

Anclar la salida en la realidad (grounding)

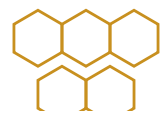
Especializar no sirve si el agente inventa. El **grounding** ata cada afirmación a una fuente verificable —el documento del que salió, el dato real—, para que el agente pueda mostrar *de dónde* sacó cada cosa. Es el cimiento de la confianza en entornos serios.

> PARA RECORDAR
>
> Adapta de lo ligero a lo profundo: ejemplos en contexto → RAG → fine-tuning. Empieza siempre por RAG (barato y sin tocar el modelo) y ancla las respuestas en fuentes verificables. Entrena solo cuando el resto se quede corto.

PARTE DOS

Arquitectura y patrones de diseño

El corazón del libro: un catálogo de patrones probados para construir, coordinar y endurecer sistemas de agentes.



CAPÍTULO CUATRO

Arquitectura de un sistema agéntico

> *Define bien tus piezas y la mitad del diseño está hecho.*
>
> — Sobre los componentes y sus relaciones

Antes del catálogo de patrones, fijemos el vocabulario: qué es exactamente un agente, de qué está hecho y cómo se relaciona con otros.

Tres cosas que se confunden

Un **LLM** completa texto. Un **workflow automatizado** ejecuta pasos fijos que tú definiste. Un **agente** decide por sí mismo qué paso dar para lograr un objetivo. La frontera está en quién toma las decisiones: en el workflow, tú; en el agente, el modelo dentro de un bucle.

La anatomía del agente

Un agente tiene un **modelo** (razona), **instrucciones** (su rol y reglas), **herramientas** (sus manos), **memoria** (contexto y largo plazo) y un **bucle de control** (orquesta los pasos). Dos casos lo ilustran: un agente de planificación de viajes (busca, compara, reserva) y un sistema de procesamiento de préstamos (recibe la solicitud, verifica datos, evalúa riesgo, decide), que recorreremos a fondo en la Parte 3.



Figura 4.1 — Anatomía de un agente y su entorno.

Modelos de interacción y A2A

Los agentes se relacionan de varias formas: con el usuario, con herramientas, y —clave para este libro— **entre ellos**. La comunicación agente-a-agente (**A2A**) es lo que convierte una colección de agentes en un sistema. Cómo estructurar esa comunicación es, precisamente, lo que resuelven los patrones de los siguientes capítulos.

PARA RECORDAR

LLM (completa) ≠ workflow (pasos fijos) ≠ agente (decide). Un agente es modelo + instrucciones + herramientas + memoria + bucle, inmerso en datos y entorno. Y A2A — que los agentes se hablen— es lo que vuelve un sistema a un montón de agentes.

CAPÍTULO CINCO

Patrones de coordinación multiagente

> *Coordinar no es mandar a todos: es decidir quién decide.*
>
> — El problema central del trabajo en equipo

Aquí empieza el catálogo. Cuando varios agentes deben colaborar, surge la pregunta de *cómo* se reparten el trabajo y resuelven sus diferencias. Estos son los patrones probados, con su problema, su solución y cuándo usarlos.

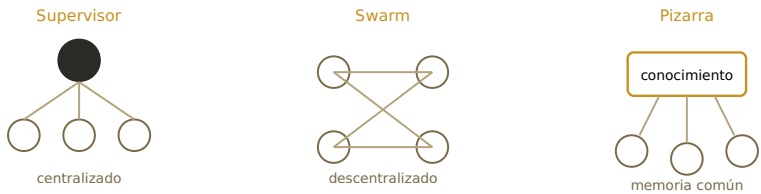


Figura 5.1 — Tres formas de organizar la colaboración: jefe central, pares libres, o pizarra compartida.

Agent Router (enrutamiento por intención)

PROBLEMA Una entrada puede necesitar a distintos especialistas y no sabes a cuál enviarla.

SOLUCIÓN Un agente "recepcionista" clasifica la intención y deriva al agente adecuado.

CUÁNDO Tienes varios agentes especializados y consultas variadas (p. ej., enrutar una solicitud de cumplimiento).

Supervisor (orquestación centralizada)

PROBLEMA Necesitas control y trazabilidad sobre un proceso de varios pasos.

SOLUCIÓN Un supervisor reparte sub-tareas a trabajadores y reúne los resultados.

CUÁNDO Procesos claros y auditables (p. ej., procesar un préstamo de forma centralizada).

Swarm (coordinación descentralizada emergente)

PROBLEMA Un control central es un cuello de botella o un punto único de fallo.

SOLUCIÓN Agentes con reglas locales que coordinan sin jefe; el orden emerge.

CUÁNDO Exploración masiva o tareas paralelas sin orden estricto (p. ej., creación de contenido distribuida).

Pizarra (Blackboard) — centro de conocimiento

PROBLEMA Varios especialistas deben aportar a un problema común sin pisarse.

SOLUCIÓN Una "pizarra" compartida donde cada agente lee y escribe su aporte.

CUÁNDO Problemas que requieren múltiples expertises (p. ej., diagnóstico médico colaborativo).

Contract-Net (mercado de contratos)

PROBLEMA ¿Qué agente es el mejor para una tarea concreta, aquí y ahora?

SOLUCIÓN Un mediador publica la tarea; los agentes "ofertan" y gana el más apto.

CUÁNDO Asignación dinámica según capacidad o carga (p. ej., elegir un proveedor de nube).

Otros patrones de coordinación

ÁRBOL DE SUPERVISIÓN jerarquía con capacidades acotadas y resiliencia (p. ej., un scraper robusto).

PLANIFICACIÓN MULTIAGENTE varios agentes construyen un plan conjunto (informe de mercado).

COMPARTIR CONOCIMIENTO reutilizar soluciones halladas por otros agentes.

ENRUTAMIENTO DE HERRAMIENTAS dirigir cada llamada a la herramienta correcta entre muchas.

CONSENSO los agentes debaten y convergen en una decisión (pronóstico financiero).

NEGOCIACIÓN acordar el reparto de un recurso compartido.

ASIGNACIÓN DE RECURSOS repartir recursos escasos (robots en una fábrica).

RESOLUCIÓN DE CONFLICTOS jerárquica, por política, por negociación o teoría de juegos — siempre con detección, auditoría y escalado a humano.

CONTROL DE FORMACIÓN mantener una estructura coordinada en movimiento (enjambre de drones agrícolas).

PARA RECORDAR

La coordinación es un dial entre *centralizado* (Supervisor: control y trazabilidad) y *descentralizado* (Swarm: robustez y escala). Entre medias hay piezas para repartir tareas (Router, Contract-Net), compartir (Pizarra) y resolver diferencias (Consenso, Negociación, Conflictos). Elige por el grado de control que el proceso exige.

CAPÍTULO SEIS

Patrones de explicabilidad y cumplimiento

> *En la empresa no basta con acertar: hay que poder demostrar por qué se acertó.*
>
> — Sobre la diferencia entre listo y confiable

En entornos regulados —banca, salud, legal— un agente debe ser **explicable** (poder mostrar su razonamiento) y **conforme** (respetar reglas que no puede saltarse). Estos patrones lo logran.

Auditoría de fidelidad a las instrucciones

PROBLEMA ¿Cómo saber si el agente realmente siguió las reglas que le diste?

SOLUCIÓN Un mecanismo que verifica, paso a paso, que la salida respeta las instrucciones.

CUÁNDO Reglas que no se pueden violar (p. ej., auditar la aplicación de un descuento en e-commerce).

Cadena de pensamiento fractal (embebida)

PROBLEMA El razonamiento de tareas complejas se vuelve opaco y difícil de auditar.

SOLUCIÓN Estructurar el razonamiento en niveles anidados, cada uno explicable por sí mismo.

CUÁNDO Síntesis o investigación colaborativa donde hay que rastrear cada conclusión.

Anclaje persistente de instrucciones

PROBLEMA En conversaciones largas, el agente "olvida" restricciones críticas dadas al inicio.

SOLUCIÓN Re-inyectar las restricciones clave de forma constante para que nunca se diluyan.

CUÁNDO Restricciones que deben mantenerse siempre (p. ej., reglas en un reporte financiero).

Memoria epistémica compartida

PROBLEMA Varios agentes deben coincidir en *qué se sabe y con qué certeza*.

SOLUCIÓN Una memoria común que registra hechos, su fuente y su nivel de confianza.

CUÁNDO Decisiones colectivas sobre datos cambiantes (p. ej., una disrupción en la cadena de suministro).

Componer patrones para fiabilidad sistémica

Como en el panal, estos patrones rara vez van solos: se **componen**. Auditar la fidelidad + anclar las instrucciones + una memoria epistémica común producen un sistema que no solo decide bien, sino que puede *demostrar* por qué decidió así. Esa demostrabilidad es, en la empresa, tan valiosa como el acierto.

PARA RECORDAR

Explicabilidad = poder mostrar el razonamiento; cumplimiento = no poder saltarse las reglas. Se logran anclando instrucciones, auditando la fidelidad y compartiendo lo que se sabe y con qué certeza. En entornos regulados, "no puedo explicarlo" equivale a "no puedo usarlo".



CAPÍTULO SIETE

Patrones de robustez y tolerancia a fallos

> *No preguntes si tu agente fallará, sino qué pasará cuando lo haga.*
>
> — La mentalidad de producción

Las herramientas caen, las redes se cortan, los modelos se confunden. Un sistema serio no evita los fallos —imposible—, sino que los **sobrevive**. La robustez es un espectro de niveles, y estos patrones te suben por él.

Consenso por ejecución paralela

PROBLEMA Una sola respuesta del modelo puede ser un error puntual.

SOLUCIÓN Ejecutar la tarea varias veces en paralelo y comparar; la coincidencia da confianza.

CUÁNDO Decisiones críticas (p. ej., validar una evaluación de riesgo crediticio).

Voto mayoritario entre agentes

PROBLEMA Necesitas una decisión final fiable a partir de juicios que pueden discrepar.

SOLUCIÓN Varios agentes deciden; gana la mayoría.

CUÁNDO Decisiones de alto impacto (p. ej., aprobar o no un préstamo).

Watchdog (supervisor de tiempos)

PROBLEMA Un agente se queda colgado y bloquea todo el proceso.

SOLUCIÓN Un temporizador vigilante que corta y reintenta si algo tarda demasiado.

CUÁNDO Cualquier paso que pueda colgarse (p. ej., un análisis que no responde).

Reintento adaptativo con mutación del prompt

PROBLEMA Reintentar igual da el mismo error.

SOLUCIÓN Al reintentar, ajustar el prompt o el enfoque según por qué falló.

CUÁNDO Tareas frágiles como extracción de datos de formatos variables.

Auto-sanación y checkpoints

PROBLEMA Un agente se cae a mitad de un proceso largo y se pierde todo el avance.

SOLUCIÓN **Auto-sanación**: detectar la caída y reiniciar el agente. **Checkpoints**: guardar el progreso por etapas para reanudar desde el último punto bueno.

CUÁNDO Pipelines largos de varias etapas (p. ej., procesar documentos por fases).

Escalado diferido y grafo de dependencias causales

PROBLEMA Escalar al humano demasiado pronto lo satura; demasiado tarde, falla el caso.

SOLUCIÓN El **escalado diferido** agota primero las opciones automáticas y escala solo si la confianza es baja. El **grafo causal** registra qué decisión dependió de qué, para auditar y rastrear el origen de un fallo.

CUÁNDO Verificaciones de confianza variable (p. ej., un control de cumplimiento dudoso).

Defensa del agente y malla de defensa

PROBLEMA Un atacante intenta manipular al agente (p. ej., con inyección de prompts).

SOLUCIÓN **Auto-defensa**: el agente detecta y neutraliza entradas maliciosas. **Malla de defensa**: los agentes se protegen mutuamente, sin un único punto débil.

CUÁNDO Agentes expuestos a contenido externo no confiable.

ENCADENAR PATRONES

En producción rara vez usas uno solo. Un préstamo real podría: validar con consenso paralelo → vigilar con watchdog → reintentar con mutación si falla → hacer checkpoint de cada etapa → decidir por voto mayoritario → registrar todo en un grafo causal. Los patrones se encadenan como celdas del panal.

PARA RECORDAR

La robustez no evita fallos: los sobrevive. Vigila (watchdog), reintenta con inteligencia, guarda el progreso (checkpoints), decide por consenso o mayoría, y defiéndete de ataques. Mide tu nivel de robustez y sube un patrón a la vez.



CAPÍTULO OCHO

Patrones de interacción humano–agente

> *El mejor sistema no reemplaza al humano: sabe exactamente cuándo llamarlo.*
>
> — Sobre el trabajo en equipo persona–agente

Por más autónomo que sea, un agente empresarial vive entrelazado con personas que lo dirigen, lo revisan y responden por él. Estos patrones diseñan esa relación.

El humano en el bucle

El patrón fundamental: para acciones de alto impacto o decisiones inciertas, el agente **prepara** la acción y una persona la **aprueba**. No es desconfianza; es repartir bien el trabajo —velocidad de máquina, criterio humano—. La clave de diseño es *cuándo* involucrar a la persona: ni en todo (la satura) ni en nada (pierdes el control).

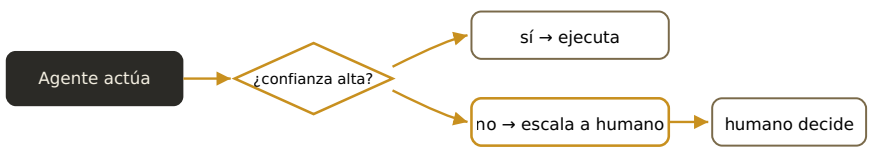


Figura 8.1 — Escalado por confianza: el agente resuelve lo seguro y pasa al humano lo dudoso.

Comunicar confianza y rendir cuentas

Un buen agente dice **cuán seguro está**: distingue "esto es seguro" de "esto es mi mejor estimación". Y el sistema define con claridad **quién responde** cuando algo sale mal, con un rastro

auditable de qué decidió el agente y qué decidió la persona. La confianza del negocio se construye sobre esa transparencia.

PARA RECORDAR

Diseña el *cuándo* del humano: que el agente resuelva lo rutinario y escale lo incierto o lo grave. Que comunique su nivel de confianza y que siempre quede claro quién rinde cuentas. El objetivo no es autonomía máxima, sino el mejor equipo persona–agente.



CAPÍTULO NUEVE

Patrones a nivel de agente

> *Un buen equipo empieza por un buen jugador. Antes de coordinar agentes, construye uno que valga la pena coordinar.*
>
> — Sobre la calidad individual

Antes de pensar en sistemas, conviene que *cada* agente esté bien hecho. Estos patrones aplican a la pieza individual, sea que trabaje sola o en equipo.

El agente individual bien construido

- **Instrucciones claras y un rol único**: un agente que hace una cosa y la hace bien es más fiable que uno que intenta todo. La especialización es una virtud, no una limitación.
- **Razonar antes de actuar**: que piense por pasos (cadena de pensamiento) antes de usar una herramienta reduce errores notablemente.
- **Reflexión**: que revise su propio trabajo antes de entregarlo —releer el correo antes de enviarlo—.
- **Manejo de incertidumbre**: que sepa decir "no estoy seguro" y pedir datos en vez de inventar.
- **Herramientas bien descritas y validadas**: el agente acierta en la medida en que sus herramientas están bien explicadas y sus entradas, verificadas.

ANALOGÍA

Un agente a nivel individual es como un buen empleado: rol claro, piensa antes de actuar, revisa su trabajo, sabe cuándo pedir ayuda y usa bien sus herramientas. Ningún organigrama salva a una empresa de empleados que no dominan su puesto.

PARA RECORDAR

Antes de orquestar, perfecciona la pieza: rol único, razonamiento por pasos, auto-revisión, honestidad ante la duda y herramientas bien descritas. Un sistema multiagente amplifica la calidad de sus agentes —y también sus defectos.

CAPÍTULO DIEZ

Patrones a nivel de sistema para producción

> *Funciona en mi laptop no es una arquitectura.*
>
> — Sobre lo que separa una demo de un producto

Los patrones anteriores construyen agentes y los coordinan. Estos los preparan para vivir en producción: observados, gobernados y listos para escalar.

Observabilidad

No puedes operar lo que no ves. Cada agente debe dejar **trazas**: qué recibió, qué razonó, qué herramienta usó, qué devolvió, cuánto tardó y costó. Con eso construyes tableros y alertas que te avisan de un problema antes que tus usuarios. La traza es la caja negra del sistema.

Despliegue seguro

Cambia con red de seguridad: el **modo sombra** corre la versión nueva en paralelo sin afectar a nadie, para comparar; el **despliegue canario** la libera a un pequeño porcentaje de usuarios antes de extenderla. Así un cambio malo se detecta con poco daño.



Figura 10.1 — La cadena de producción: instrumentar, visualizar, alertar, gobernar.

Gobernanza operativa

A escala, alguien debe ser **dueño** de cada métrica y de cada agente. Sin responsables, los problemas no se arreglan. Define cuotas de costo, límites de acción por agente y un proceso claro para revisar y aprobar cambios. La gobernanza no frena la innovación: la hace sostenible.

PARA RECORDAR

Producción = observabilidad (trazas, tableros, alertas) + despliegue con red (sombra y canario) + gobernanza (dueños, cuotas, límites). Construye estos cimientos antes de subir la autonomía, no después del primer incidente.



CAPÍTULO ONCE

Adaptación avanzada: agentes que aprenden

> *El sistema que no aprende de sus errores está condenado a repetirlos en producción.*
>
> — Sobre la mejora continua

Un agente estático envejece: el mundo cambia y él se queda. Los agentes avanzados **aprenden** —de su experiencia y del feedback— para mejorar con el tiempo.

Aprender de la experiencia (sin reentrenar)

La forma más práctica y barata. El agente guarda un registro de **qué funcionó y qué no** en casos pasados, y lo consulta ante uno nuevo parecido. Tras un fallo, escribe una nota —"esto salió mal porque…"— y la tiene presente la próxima vez. Mejora sin tocar el modelo, de forma rápida y auditable.

Aprender del feedback

Cada corrección de un usuario, cada pulgar abajo, es oro. Un buen sistema recoge ese feedback en un canal ordenado, encuentra la **causa raíz** con las trazas, prioriza por impacto y **cierra el ciclo**: cada fallo corregido se convierte en un nuevo caso de prueba, para que no vuelva a ocurrir sin que el sistema lo note.

Aprender en los pesos (cuando vale la pena)

Cuando acumulas muchos datos de calidad, puedes dar el paso de **afinar el modelo** para incorporar de forma permanente lo aprendido. Es potente pero costoso; resérvalo para problemas estables donde el volumen de datos lo justifique.

> PARA RECORDAR
>
> Aprender es la diferencia entre un sistema que envejece y uno que mejora. Empieza por lo barato y reversible (experiencia y feedback con ciclo cerrado) y reserva el reentrenamiento para cuando los datos lo justifiquen. Un fallo aprovechado vale más que un acierto olvidado.



PARTE TRES

Ejecución: estrategia, casos y futuro

De la teoría a la práctica: una hoja de ruta y un caso real recorrido tres veces.



CAPÍTULO DOCE

Hoja de ruta práctica por madurez

> *No adoptes todos los patrones*; *adopta el siguiente*.
>
> — Sobre crecer sin ahogarse

Con un catálogo tan grande, el riesgo es quererlo todo de golpe. Esta hoja de ruta te dice qué patrón conviene en cada nivel de madurez, para que crezcas paso a paso.

Patrones por nivel

- **Nivel 1–2 (asistente, copiloto)**: empieza con un agente único bien hecho (cap. 9) y RAG (cap. 3). Aún no necesitas coordinación.
- **Nivel 3 (agente que ejecuta)**: añade herramientas, observabilidad básica (cap. 10) y el humano en el bucle (cap. 8) para las acciones de impacto.
- **Nivel 4 (multiagente)**: introduce coordinación —empieza por el Supervisor, el más controlable (cap. 5)— y los primeros patrones de robustez (watchdog, reintentos).
- **Nivel 5–6 (auto-correctivo)**: suma consenso, voto mayoritario, auto-sanación, defensa, explicabilidad y aprendizaje. Aquí el sistema se vigila y se repara a sí mismo.

LA REGLA DE ORO DE LA HOJA DE RUTA

Cada patrón resuelve un dolor concreto. No adoptes un patrón hasta sentir el dolor que cura. Añadir robustez avanzada a un prototipo que nadie usa es como reforzar los cimientos de una casa que aún no decides construir.

PARA RECORDAR

Crece por niveles: agente único + RAG → herramientas + supervisión humana → coordinación (Supervisor) + robustez básica → consenso, defensa y aprendizaje. Adopta el siguiente patrón cuando sientas el problema que resuelve, no antes.



CAPÍTULO TRECE

Caso: un agente único para préstamos

> *Empieza con lo más simple que funcione. Casi siempre es más de lo que crees.*
>
> — Sobre el primer agente

Bajemos a tierra con un caso real que recorreremos tres veces, cada vez más sofisticado. Empezamos por lo mínimo: **un solo agente** que procesa solicitudes de préstamo.

El flujo de un agente

El agente recibe una solicitud y, en un solo bucle, hace todo: extrae los datos del solicitante, los verifica, consulta el historial crediticio (una herramienta), evalúa el riesgo según las reglas del banco y emite una recomendación. Una sola pieza, un solo cerebro, un solo flujo.



Figura 13.1 — Un agente único hace todo el flujo de principio a fin.

Sus virtudes y sus límites

Lo bueno: es **simple** de construir, depurar y entender. Para volúmenes modestos y reglas claras, es perfecto y no necesitas más. Lo malo: cuando crece la complejidad —muchos tipos de préstamo, regulaciones distintas, picos de carga— un solo

agente se vuelve un cuello de botella y un cerebro sobrecargado que mezcla demasiadas responsabilidades. Ese límite es lo que nos llevará, en el siguiente capítulo, a repartir el trabajo.

PARA RECORDAR

El agente único es el punto de partida correcto: simple, claro y suficiente para muchos casos. Construye este primero, mídelo, y solo divídelo cuando su simplicidad empiece a costarte más de lo que te ahorra.

CAPÍTULO CATORCE

Caso: un sistema multiagente para préstamos

> *Lo que un genio hace despacio, un equipo bien organizado lo hace rápido y mejor.*
>
> — Sobre repartir el trabajo

Tomamos el mismo caso del préstamo y lo rediseñamos como **varios agentes especializados** coordinados por un supervisor. Aquí vemos los patrones de la Parte 2 trabajando juntos.

El sistema en acción

Un **supervisor** (patrón del cap. 5) recibe la solicitud y la reparte: un agente *verificador* comprueba la identidad y los documentos; un agente *de riesgo* evalúa el historial crediticio; un agente *de cumplimiento* revisa que todo respete la normativa (con RAG, cap. 3). Para la decisión final crítica, varios agentes votan (**voto mayoritario**, cap. 7). Si la confianza es baja, se **escala a un humano** (cap. 8). Y todo el proceso queda en un **grafo causal** auditable (cap. 7).

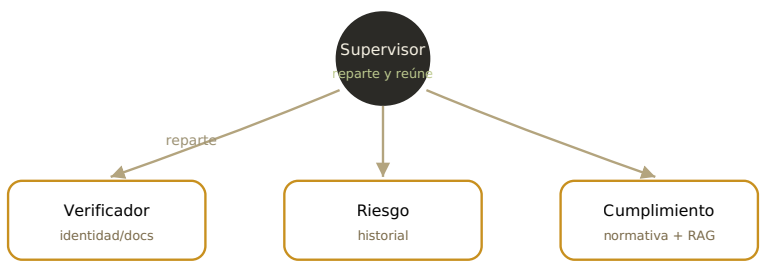


Figura 14.1 — El mismo caso, ahora como equipo de especialistas bajo un supervisor.

Qué ganamos y qué cuesta

Ganamos **especialización** (cada agente experto en lo suyo), **paralelismo** (verificación y riesgo a la vez), **robustez** (si uno falla, el sistema lo maneja) y **escalabilidad**. El costo: más llamadas al modelo, más complejidad, más cosas que coordinar y vigilar. Por eso este diseño se justifica cuando el volumen y la complejidad lo piden —no para tres solicitudes al día—.

PARA RECORDAR

El sistema multiagente convierte un cerebro sobrecargado en un equipo de especialistas coordinados. Aquí los patrones se componen como celdas de panal: supervisor + RAG + voto mayoritario + escalado humano + grafo causal. Más poder y robustez, a cambio de más costo y coordinación.

CAPÍTULO QUINCE

El mismo caso con frameworks

> *El framework correcto te ahorra el trabajo aburrido; no te ahorra el diseño.*
>
> — Sobre las herramientas del oficio

Construir todo a mano enseña, pero en producción conviene apoyarse en frameworks que resuelven lo repetitivo. Veamos nuestro sistema de préstamos apoyado en ellos.

Qué aporta un framework

Un framework de agentes te da andamios listos: el bucle de control, la gestión del estado, la comunicación entre agentes, la integración de herramientas, la observabilidad. En vez de programar la fontanería, describes *qué* agentes quieres y *cómo* se coordinan. Herramientas como las de orquestación de grafos, las de equipos de agentes o los SDKs de los propios proveedores cubren buena parte de los patrones de este libro de fábrica.

```
// Idea de cómo se ve definir el sistema con un framework (pseudocó-
digo)
const verificador  = crearAgente({ rol: "verifica identidad y docu-
mentos", tools:[docs] });
const riesgo       = crearAgente({ rol: "evalúa riesgo crediticio",
tools:[buroCredito] });
const cumplimiento = crearAgente({ rol: "revisa normativa", tools:
[ragNormas] });

const supervisor = crearSupervisor({
  trabajadores: [verificador, riesgo, cumplimiento],
  decision: "voto_mayoritario",
  escalado: "a_humano_si_confianza_baja"
});

await supervisor.procesar(solicitudDePrestamo);  // el framework or-
questa todo
```

La advertencia importante

Un framework acelera, pero **no piensa por ti** . Las decisiones de diseño —qué agentes, cómo se coordinan, qué patrones de robustez, dónde entra el humano— siguen siendo tuyas. El framework es la cera; el panal lo diseñas tú. Por eso aprendimos los patrones *antes* de hablar de frameworks: para usarlos con criterio y no quedar atrapados en lo que cada uno hace fácil.

PARA RECORDAR

Los frameworks te ahorran la fontanería (bucle, estado, comunicación, observabilidad) y traen muchos patrones de fábrica. Pero el diseño —qué patrones, cómo se componen— sigue siendo decisión humana. Entiende los patrones primero; el framework, después.

CAPÍTULO DIECISÉIS

Conclusión: traza tu camino agéntico

> *No construyas el panal entero de golpe. Construye una celda perfecta, y luego la siguiente.*
>
> — La lección de la abeja

Hemos recorrido el viaje completo: del modelo al agente, del agente al sistema, y del sistema a la producción. Cerremos con la brújula para tu propio camino.

Lo esencial en cinco ideas

- **Mapea tu madurez** y crece por niveles; no adoptes patrones que no necesitas aún.
- **Empieza por un agente único bien hecho** y RAG; divide solo cuando la complejidad lo pida.
- **Los patrones se componen** como celdas de panal: coordinación + explicabilidad + robustez juntos hacen un sistema confiable.
- **Producción es observabilidad**, **gobernanza y el humano en el bucle**, no solo más agentes.
- **Diseña tú**; **deja que el framework ejecute**. La cera la pone la herramienta; la forma, tú.

La lección de la abeja

Vuelve a la portada. La colmena no se construye de una vez ni por un arquitecto genial: se levanta celda a celda, cada una con la misma forma probada, hasta formar una estructura

asombrosamente fuerte y eficiente. Tu sistema de agentes se construye igual. No necesitas todos los patrones de este libro hoy; necesitas reconocer, cuando aparezca el problema, qué celda encaja. Empieza con una celda perfecta —un agente que de verdad funcione— y construye desde ahí. El resto es saber qué patrón añadir, y cuándo.

PARA RECORDAR

Tienes el catálogo completo. Ahora la disciplina: madurez antes que ambición, una celda antes que el panal entero, composición antes que complejidad, y diseño humano antes que framework. Traza tu camino una celda a la vez.

Glosario

- **Patrón de diseño**: solución probada y reutilizable a un problema recurrente.
- **Agente vs workflow**: el agente decide los pasos; el workflow los tiene fijos.
- **A2A**: comunicación agente-a-agente, lo que vuelve un sistema a un grupo de agentes.
- **RAG / fine-tuning / ICL**: formas de adaptar el modelo, de ligera a profunda.
- **AgentOps**: operar LLMs agénticos de forma medible, versionada y reversible.
- **Supervisor / Swarm**: coordinación centralizada vs descentralizada.
- **Pizarra / Contract-Net**: conocimiento compartido vs asignación por ofertas.

- **Consenso / voto mayoritario**: robustez por acuerdo entre varios agentes.
- **Watchdog / checkpoints / auto-sanación**: sobrevivir a cuelgues y caídas.
- **Grafo causal**: registro auditable de qué decisión dependió de qué.
- **Human-in-the-loop**: la persona aprueba o decide lo de alto impacto.
- **Modo sombra / canario**: probar cambios en producción con poco riesgo.
- **Grounding**: atar cada afirmación a una fuente verificable.

Fin · Patrones Arquitectónicos para Sistemas Multiagente · Edición de autor

